UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

ESTRUCTURA DE DATOS II

SISTEMA DE RED DE TRANSPORTE URBANO - PROYECTO 2

"MANUAL TECNICO"

INTEGRANTE:	**CARNET:**
DIEGO ANGEL ALBERTO BARRIOS AJQUILL	1557125
MANUEL ALEJANDRO COXAJ RAMÍREZ	1506925

QUETZALTENANGO, NOVIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTAD

1. Arquitectura del Sistema

El proyecto sigue un patrón de diseño modular que separa claramente los datos, la lógica de estructuras y la interfaz gráfica.

- Proyecto2Estructuras.Modelos: Contiene las clases base de datos (Parada, Ruta, ResultadoRuta). Son objetos puros de transferencia de datos (DTO).
- Proyecto2Estructuras.Estructuras: Es el motor del programa. Contiene las implementaciones manuales de los TDA (Tipos de Datos Abstractos).
- Proyecto2Estructuras.Datos: Gestores de entrada/salida (CargadorParadas, CargadorRutas). Se encargan de la lectura y serialización hacia y desde archivos CSV.

2. Estructuras de Datos Implementadas

Para cumplir con los requerimientos técnicos, se desarrollaron las siguientes estructuras desde sus fundamentos teóricos:

1.1 Grafo:

Representa la red de transporte, estructura escogida para la organización de las paradas y rutas gracias a la conectividad entre vértices al poder ser tratados como dirigidos o no dirigidos (una vía o doble vía respectivamente según sea el caso).

- Utiliza una lista simplemente enlazada de Vértice (representando las paradas) donde cada vértice contiene un puntero a su propia lista enlazada de Ruta (aristas).
- Las aristas se insertan manteniendo un orden alfabético por destino para optimizar recorridos y búsquedas.
- Búsqueda de vértice $O(V)$, inserción de arista $O(E)$.

2.1 Tabla Hash

Actúa como la base de datos principal de acceso y almacenamiento rápido para la información de las paradas, esto gracias a su facilidad de búsqueda y guardado de datos según la función hash.

- Arreglo estático con una función hash basada en el valor ASCII de los caracteres del código de la parada.
- Para garantizar que los códigos únicos (P001) se almacenen y recuperen en tiempo $O(1)$ promedio.

3.1 Trie (Árbol de Prefijos)

Es el encargado de buscar paradas de los autobuses por medio de la autocomprobación del nombre, estructura escogida gracias a la facilidad de la conexión de los datos con ayuda del nombre y el código por medio de la tabla hash.

- Cada nodo (NodoTrie) contiene un arreglo de 256 posiciones (soportando todo el mapa ASCII básico) y una estructura dinámica personalizada (ListaCadenas) que almacena los formatos Código|Nombre.
- Búsqueda en tiempo $O(L)$, donde L es la longitud del prefijo ingresado.

4.1 Union-Find (Conjuntos Disjuntos)

Estructura auxiliar para validación de conectividad.

- Arreglo de enteros que representa bosques de árboles. Se utiliza para determinar si dos paradas pertenecen al mismo componente conexo antes de ejecutar algoritmos costosos como Dijkstra.

3. Algoritmos de Grafos

La clase AlgoritmosGrafo consolida la lógica matemática de la red:

1. Búsqueda en Anchura (BFS): Utiliza una cola (Queue<string>) para recorrer el grafo nivel por nivel desde un nodo de inicio, retorna un arreglo primitivo con el orden exacto de visita.
2. Búsqueda en Profundidad (DFS): Implementado mediante recursividad, explorando cada rama del grafo hasta su máxima profundidad antes de retroceder (backtracking).
3. Dijkstra:
    - No utiliza diccionarios; emplea arreglos paralelos (distancias[], costos[], tiempos[], anteriores[], visitados[]) indexados matemáticamente al arreglo de vértices.
    - Tie-Breaker: Si el algoritmo optimiza por *Tiempo* y encuentra dos rutas que duran exactamente lo mismo, la lógica interna evalúa la matriz de *Costos* y elige la más económica.
    - Calcula simultáneamente el impacto secundario para ofrecer reportes completos (ej. Ruta más rápida + cuánto costó).

DIAGRAMA DE FLUJO:

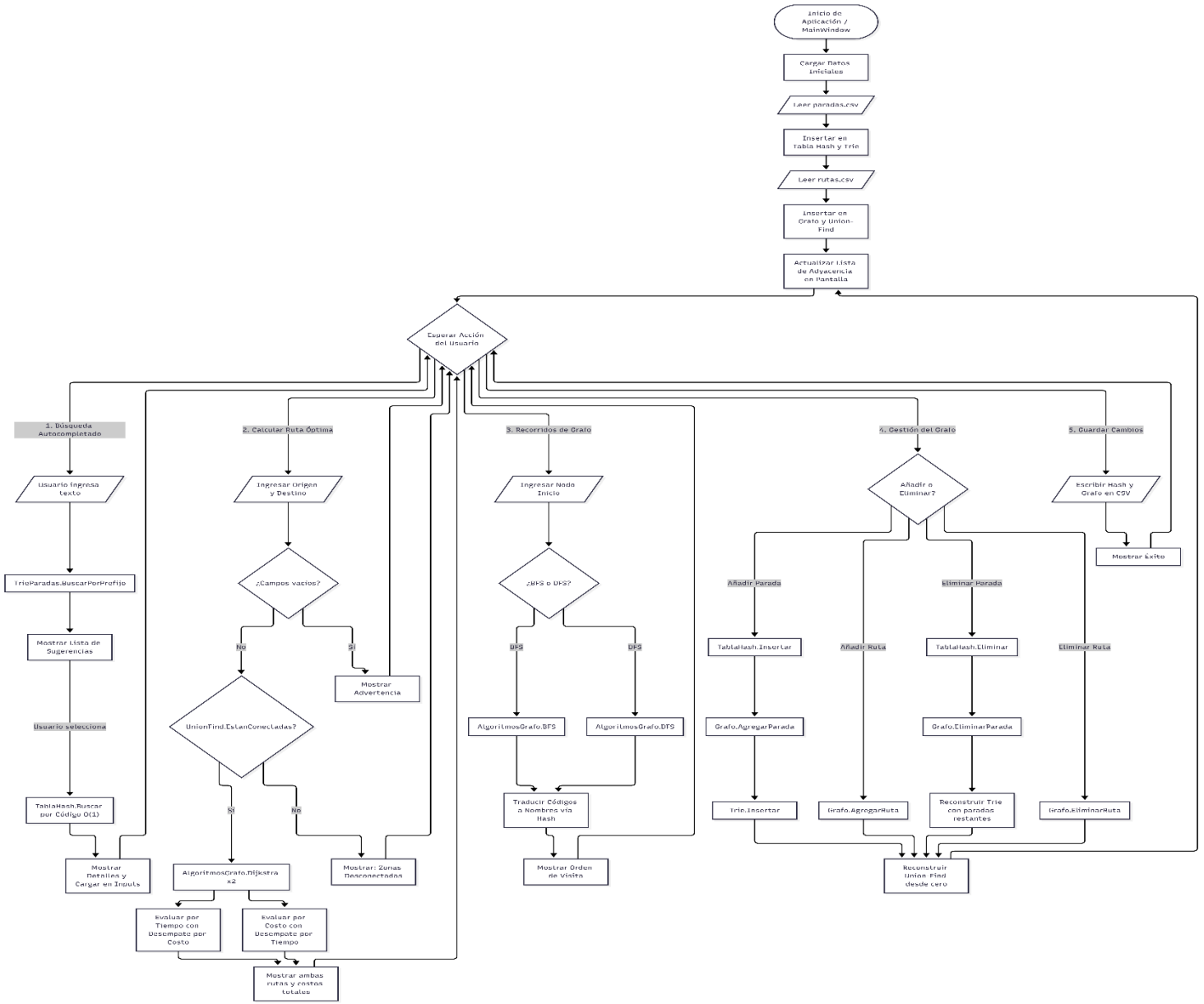


DIAGRAMA DE CLASES:

